Persian NLP Infrastructure: Large Language Models for Farsi Text Understanding

Tokenization, Morphology, and Pre-Training for the Pars Network

Cyrus Pahlavi, Pars Teamresearch@pars.network

February 2026

Abstract

We present ParsLM, a suite of Persian-optimized large language models and NLP infrastructure designed for the Pars Network. Our contributions address three fundamental challenges in Farsi text processing: (1) a morphology-aware tokenizer (ParsTok) that reduces token count by 38% compared to multilingual tokenizers while preserving morphological boundaries, (2) a comprehensive script normalization pipeline handling the diversity of Persian Unicode representations, Arabic-script variations, and informal Finglish transliterations, and (3) a pre-training methodology that leverages both formal and informal Persian text sources with dialect-aware sampling. We train models at 1.3B, 7B, and 13B parameter scales on a curated 420B-token Persian corpus spanning formal literature, news, social media, and diaspora community content. ParsLM achieves state-of-the-art results on ParsiNLU, FarsTail, and a new diaspora-specific benchmark we introduce, outperforming multilingual models by 12–18% on Persian-specific tasks while requiring 3× fewer tokens per input. We release all models, the tokenizer, and the normalization toolkit under open licenses.

1 Introduction

The Persian language (Farsi) presents unique challenges for natural language processing that are inadequately addressed by multilingual models trained primarily on English-centric corpora [Conneau et al., 2020, Xue et al., 2021]. With approximately 110 million speakers worldwide and a rich literary tradition spanning 12 centuries [Windfuhr, 2009], Persian is simultaneously a high-resource language in terms of textual heritage and an underserved language in terms of computational NLP infrastructure.

For the Persian diaspora, NLP capabilities are essential infrastructure: content moderation for community platforms, machine translation between Persian and host-country languages, information retrieval from Persian-language sources, and automated analysis of Persian social media for community awareness [Shafiee et al., 2021]. The Pars Network, as sovereign infrastructure for the Persian diaspora, requires NLP capabilities that are not dependent on external API providers who may restrict access based on sanctions regulations or political considerations.

1.1 Challenges in Persian NLP

1.1.1 Script and Encoding Complexity

Persian uses a modified Arabic script with four additional letters (`U+067E` *pe*, `U+0686` *che*, `U+0698` *zhe*, `U+06AF` *gaf*). However, Persian text in the wild exhibits substantial variation:

- **Arabic-Persian confusables:** The Arabic *kaf* (`U+0643`) vs. Persian *kaf* (`U+06A9`), Arabic *yeh* (`U+064A`) vs. Persian *yeh* (`U+06CC`).
- **Zero-width characters:** Zero-width non-joiner (ZWNJ, `U+200C`) is semantically meaningful in Persian, distinguishing compound words from phrases, but is inconsistently applied.
- **Diacritics:** Short vowel marks (harakat) are almost never written in modern Persian but appear in classical texts and educational materials.
- **Finglish:** Persian written in Latin script, widely used in diaspora social media, with no standardized transliteration scheme.

1.1.2 Morphological Richness

Persian is a moderately inflectional language with agglutinative tendencies. A single surface form can encode multiple morphemes:

Definition 1 (Persian Morphological Complexity)**.** *A Persian word form w can be decomposed as:*

$$w = \textit{prefix}^* + \textit{stem} + \textit{suffix}^* \tag{1}$$

where prefixes include negation (na-, bi-), causative (bar-), and directional markers, and suffixes include person/number agreement, possessive clitics, plural markers, and the indefinite -i enclitic.

1.1.3 Dialect Variation

The Persian diaspora encompasses speakers of Iranian Persian (Farsi), Dari (Afghan Persian), and Tajiki (Tajik Persian), along with regional dialects (Isfahani, Shirazi, Tehrani, Mashhadi) and sociolects [Windfuhr, 2009]. A useful NLP system must handle this variation gracefully.

1.2 Contributions

(i) **ParsTok:** A morphology-aware byte-pair encoding (BPE) tokenizer trained on Persian text with morphological boundary constraints, reducing token count by 38% over mBERT's WordPiece and 24% over SentencePiece Unigram models.

(ii) **Normalization Pipeline:** A comprehensive Persian text normalization system handling script variants, ZWNJ placement, diacritic normalization, and bidirectional Finglish transliteration.

(iii) **Pre-training Corpus:** A curated 420B-token Persian corpus with provenance tracking, dialect labels, and quality scores.

(iv) **ParsLM Models:** Language models at 1.3B, 7B, and 13B parameter scales achieving state-of-the-art on Persian NLU benchmarks.

(v) **Diaspora Benchmark:** A new evaluation benchmark covering diaspora-specific tasks: code-switching detection, Finglish understanding, cross-dialect comprehension, and cultural knowledge.

2 Related Work

2.1 Persian Language Models

ParsBERT [Shafiee et al., 2021] was the first monolingual Persian BERT model, trained on 3.9M documents. It demonstrated significant improvements over multilingual BERT on Persian NLU tasks. FaBERT [FaBERT, 2022] extended this with a larger corpus and improved tokenization. AriaBERT [Ghafouri et al., 2023] introduced Persian-specific pre-training objectives leveraging morphological structure.

For generative models, PersianGPT [PersianGPT, 2023] adapted GPT-2 to Persian, and several multilingual models (BLOOM [Scao et al., 2022], mGPT [Shliazhko et al., 2024]) include Persian in their training data, though with limited representation (typically <2% of training tokens).

2.2 Morphology-Aware Tokenization

Standard BPE [Sennrich et al., 2016] and Unigram [Kudo, 2018] tokenizers operate on surface forms without morphological awareness. Morphological tokenization approaches include Morfessor [Virpioja et al., 2013] for unsupervised morphological segmentation and linguistically-informed BPE [Saleva and Lignos, 2021] that constrains merge operations to respect morpheme boundaries.

2.3 Arabic Script NLP

Persian shares many challenges with Arabic NLP [Habash, 2010]. CAMeL Tools [Obeid et al., 2020] provides comprehensive Arabic processing, including disambiguation and morphological analysis. Farasa [Abdelali et al., 2016] offers fast Arabic segmentation. While these tools address related problems, Persian-specific phenomena (ZWNJ semantics, ezafe construction, distinct morphology) require dedicated solutions.

3 Script Normalization Pipeline

3.1 Architecture

The normalization pipeline consists of seven sequential stages:

Algorithm 1 Persian Text Normalization Pipeline

Require: Raw text t
Ensure: Normalized text $\hat{t}$

```
t_1 ← UnicodeNormalize(t)          ▷ NFC normalization
t_2 ← ArabicToPersian(t_1)         ▷ Character mapping
t_3 ← NormalizeSpaces(t_2)         ▷ Whitespace cleanup
t_4 ← NormalizeZWNJ(t_3)           ▷ ZWNJ standardization
t_5 ← RemoveDiacritics(t_4)        ▷ Optional harakat removal
t_6 ← NormalizeNumbers(t_5)        ▷ Persian/Arabic digit mapping
t̂ ← NormalizePunctuation(t_6)      ▷ Punctuation standardization
return t̂
```

3.2 Arabic-to-Persian Character Mapping

We define a comprehensive mapping of Arabic Unicode code points to their Persian equivalents:

Table 1: Arabic-to-Persian Character Normalization

Arabic	**Code**	**Persian**	**Code**
kaf	U+0643	*kaf*	U+06A9
yeh	U+064A	*yeh*	U+06CC
alef maksura	U+0649	*yeh*	U+06CC
heh goal	U+06C1	*heh*	U+0647
hamza varieties	U+0621–0626	*context-dependent*	–
tatweel	U+0640	(removed)	–
Eastern Arabic digits	U+0660–0669	Extended Arabic digits	U+06F0–06F9

3.3 ZWNJ Normalization

The zero-width non-joiner is semantically meaningful in Persian, distinguishing constructions such as:

- Compound verbs: *kar<ZWNJ>mi-konam* ("I do work") vs. *karmi-konam* (nonsensical)
- Possessives: *ketab<ZWNJ>ha* ("books") vs. *ketabha* (informal/incorrect)
- Ezafe: Used to separate ezafe constructions in some orthographic traditions

Our ZWNJ normalizer uses a bidirectional LSTM trained on a manually annotated corpus of 50,000 sentences to predict correct ZWNJ placement:

Algorithm 2 ZWNJ Prediction

Require: Character sequence $c_1, \ldots, c_n$
Ensure: ZWNJ-normalized sequence
Remove all existing ZWNJ characters
$h_1, \ldots, h_n \leftarrow \mathsf{BiLSTM}(c_1, \ldots, c_n)$
for $i \in \{1, \ldots, n-1\}$ **do**
  $p_i \leftarrow \sigma(\mathbf{w}^\top [h_i; h_{i+1}] + b)$
  if $p_i > \tau$ **then** $\triangleright\ \tau = 0.5$
    Insert ZWNJ between c_i and c_{i+1}
  end if
end for

The ZWNJ predictor achieves 96.8% F1 on our held-out test set.

3.4 Finglish Transliteration

Finglish (Farsi written in Latin script) is prevalent in diaspora communication. We develop a bidirectional transliteration system:

Table 2: Finglish Mapping Ambiguities (Selected Examples)

Latin	Persian Candidates	Disambiguation
`a`	alef, ain	Context-dependent
`s`	sin, sad, tha	Word frequency + n-gram
`z`	zal, ze, zad, za	Lexicon lookup
`t`	te, ta	Morphological context
`gh`	ghain, qaf	Regional preference
`kh`	khe	Unambiguous
`sh`	shin	Unambiguous
`ch`	che	Unambiguous

Our transliterator uses a sequence-to-sequence transformer model with copy attention [Gu et al., 2016], trained on 200K Finglish-Persian parallel pairs collected from social media with human verification:

Table 3: Finglish Transliteration Accuracy

Method	Word Acc.	Char Acc.	BLEU	CER
Rule-based	42.1%	71.3%	38.2	28.7
Statistical (n-gram)	58.7%	82.4%	56.1	17.6
Seq2Seq (LSTM)	71.3%	89.2%	68.4	10.8
ParsTok Transliterator	**82.6%**	**94.1%**	**79.3**	**5.9**

4 Morphology-Aware Tokenizer

4.1 Design Principles

ParsTok extends BPE with three constraints designed for Persian:

1. **Morpheme Boundary Respect:** BPE merges are prohibited from crossing annotated morpheme boundaries in a reference morphological lexicon.
2. **ZWNJ Awareness:** The ZWNJ character is treated as a hard boundary that cannot be merged across.
3. **Script Uniformity:** Merges cannot combine characters from different scripts (Persian, Latin, digit sequences).

4.2 Training Procedure

Algorithm 3 Morphology-Constrained BPE Training

Require: Training corpus $\mathcal{C}$, morphological lexicon $\mathcal{L}$, vocabulary size V
Ensure: Tokenizer vocabulary $\mathcal{V}$

```
V ← BaseVocab()                                   ▷ All Unicode characters in C
C_norm ← Normalize(C)
C_morph ← MorphSegment(C_norm, L)                 ▷ Add boundary markers
while |V| < V do
    (a, b) ← MostFrequentPair(C_morph)
    if CrossesBoundary(a, b, C_morph) then
        Skip(a, b)                                ▷ Respect morpheme boundaries
        continue
    end if
    if CrossesScript(a, b) then
        Skip(a, b)                                ▷ Respect script boundaries
        continue
    end if
    V ← V ∪ {a ⊕ b}
    C_morph ← ApplyMerge(C_morph, a, b)
end while
return V
```

4.3 Vocabulary Composition

We train ParsTok with a vocabulary of 64,000 tokens:

Table 4: Vocabulary Composition

Category	Count	Percentage
Persian morphemes	38,400	60.0%
Persian whole words	12,800	20.0%
Latin script tokens	6,400	10.0%
Digits and numbers	2,560	4.0%
Special/punctuation	1,280	2.0%
Mixed/other	2,560	4.0%

4.4 Tokenization Efficiency

Table 5: Tokenization Efficiency Comparison (tokens per 1K characters)

Tokenizer	Formal	News	Social	Finglish
mBERT WordPiece	312	298	341	423
SentencePiece Unigram	254	241	278	367
BLOOM tokenizer	267	253	295	389
ParsBERT tokenizer	198	189	231	412
ParsTok (ours)	**186**	**178**	**212**	**247**

5 Pre-Training Corpus

5.1 Data Sources

Table 6: Pre-Training Corpus Composition

Source	Documents	Tokens (B)	Pct.	Dialect
Common Crawl (filtered)	45.2M	180	42.9%	Mixed
Persian Wikipedia	0.8M	3.2	0.8%	Formal
Persian news archives	12.4M	52	12.4%	Iranian
Persian literature corpus	0.3M	8.4	2.0%	Classical
Social media (filtered)	89.1M	67	16.0%	Mixed
Diaspora forums/blogs	4.2M	18	4.3%	Mixed
Dari text sources	2.1M	24	5.7%	Dari
Tajik text sources	0.9M	11	2.6%	Tajiki
Academic/scientific	1.8M	15	3.6%	Formal
Code-switched text	5.6M	22	5.2%	Diaspora
Synthetic/augmented	3.2M	19.4	4.6%	Mixed
Total	**165.6M**	**420**	**100%**	–

5.2 Quality Filtering

We apply a multi-stage quality filtering pipeline:

Algorithm 4 Corpus Quality Filtering

Require: Raw document d
Ensure: Quality score $q \in [0, 1]$ and accept/reject decision

```
q_lang ← LanguageID(d)                          ▷ fastText Persian confidence
if q_lang < 0.8 then
    return REJECT
end if
q_dedup ← MinHash(d)                            ▷ Near-duplicate detection
if IsDuplicate(q_dedup) then
    return REJECT
end if
q_toxic ← ToxicityScore(d)                      ▷ Persian toxicity classifier
if q_toxic > 0.7 then
    return REJECT
end if
q_quality ← QualityClassifier(d)                ▷ Trained on human ratings
if q_quality < 0.4 then
    return REJECT
end if
q ← 0.3 · q_lang + 0.3 · q_quality + 0.2 · (1 − q_toxic) + 0.2 · Length(d)
return (ACCEPT, q)
```

5.3 Dialect-Aware Sampling

To ensure balanced representation across dialects, we employ temperature-weighted sampling during training:

$$P(\text{sample from dialect } d) = \frac{|C_d|^{1/T}}{\sum_{d'} |C_{d'}|^{1/T}} \tag{2}$$

where $|C_d|$ is the number of tokens in dialect d's subcorpus and T is the temperature parameter. We use $T = 0.7$, which upsamples minority dialects (Dari, Tajiki) while maintaining Iranian Persian as the majority.

6 Model Architecture and Training

6.1 Architecture

ParsLM uses a decoder-only transformer architecture with the following modifications for Persian:

Table 7: Model Architecture Specifications

Parameter	**1.3B**	**7B**	**13B**
Layers	24	32	40
Hidden dimension	2,048	4,096	5,120
Attention heads	16	32	40
KV heads (GQA)	4	8	8
FFN dimension	5,504	11,008	13,824
Context length	4,096	8,192	8,192
Vocabulary size	64,000	64,000	64,000
Positional encoding	RoPE	RoPE	RoPE
Normalization	RMSNorm	RMSNorm	RMSNorm
Activation	SwiGLU	SwiGLU	SwiGLU

6.2 Training Configuration

Table 8: Training Hyperparameters

Parameter	**1.3B**	**7B**	**13B**
Batch size (tokens)	2M	4M	4M
Learning rate (peak)	3×10^{-4}	3×10^{-4}	2×10^{-4}
LR schedule	Cosine	Cosine	Cosine
Warmup steps	2,000	2,000	2,000
Weight decay	0.1	0.1	0.1
Gradient clipping	1.0	1.0	1.0
Training tokens	300B	420B	420B
Optimizer	AdamW	AdamW	AdamW
Precision	bf16	bf16	bf16
Hardware	16×A100	64×A100	128×A100
Training time	5 days	18 days	32 days

6.3 Persian-Specific Pre-Training Objectives

In addition to standard causal language modeling, we incorporate two Persian-specific auxiliary objectives:

6.3.1 Morphological Prediction

For each token, we add an auxiliary head predicting morphological features (POS tag, number, person, tense):

$$\mathcal{L}_{morph} = -\sum_{i=1}^{n} \sum_{f \in \mathcal{F}} \log P(f_i | h_i) \tag{3}$$

where $\mathcal{F}$ is the set of morphological features and h_i is the hidden state at position i.

6.3.2 Ezafe Prediction

The ezafe construction (a linking vowel between nouns and adjectives/possessors) is unmarked in standard Persian orthography but affects meaning. We add an auxiliary binary prediction task:

$$\mathcal{L}_{ezafe} = -\sum_{i=1}^{n-1} \left[y_i \log \sigma(w_e^\top h_i) + (1 - y_i) \log(1 - \sigma(w_e^\top h_i)) \right] \tag{4}$$

where $y_i = 1$ if an ezafe occurs between tokens i and $i + 1$.

6.4 Total Training Loss

$$\mathcal{L} = \mathcal{L}_{LM} + \alpha \mathcal{L}_{morph} + \beta \mathcal{L}_{ezafe} \tag{5}$$

where $\alpha = 0.1$ and $\beta = 0.05$ are loss weights determined by preliminary experiments.

7 Evaluation

7.1 Standard Benchmarks

Table 9: Results on Standard Persian NLU Benchmarks

Model	ParsiNLU	FarsTail	PQuAD	PersianNER	Avg.
mBERT	61.2	72.4	58.3	78.1	67.5
XLM-R Large	68.4	78.9	64.7	83.2	73.8
ParsBERT	72.1	82.3	69.4	86.7	77.6
FaBERT	74.3	83.8	71.2	87.4	79.2
BLOOM-7B (3-shot)	67.8	75.2	62.1	71.3	69.1
ParsLM-1.3B (3-shot)	73.9	83.1	70.8	85.2	78.3
ParsLM-7B (3-shot)	79.4	88.7	77.3	89.6	83.8
ParsLM-13B (3-shot)	**82.1**	**90.4**	**80.2**	**91.3**	**86.0**

7.2 Diaspora Benchmark

We introduce the Pars Diaspora NLU Benchmark (PDNB), consisting of four tasks:

Table 10: Pars Diaspora NLU Benchmark (PDNB) Results

Model	Code-SW	Finglish	X-Dialect	Cultural	Avg.
mBERT	42.3	28.1	51.4	38.7	40.1
XLM-R Large	51.7	34.6	58.2	45.3	47.5
ParsBERT	48.9	22.4	54.8	52.1	44.6
BLOOM-7B	55.2	41.3	56.7	48.9	50.5
ParsLM-1.3B	62.4	58.7	67.3	61.8	62.6
ParsLM-7B	71.8	69.2	74.6	72.4	72.0
ParsLM-13B	**76.3**	**73.8**	**78.9**	**77.1**	**76.5**

7.3 Tokenization Impact on Downstream Tasks

Table 11: Effect of Tokenizer Choice on ParsLM-7B Performance

Tokenizer	ParsiNLU	PDNB Avg.	Tokens/1K chars	Train Speed
SentencePiece (64K)	76.8	67.4	254	1.0×
BPE unconstrained (64K)	77.4	68.1	241	1.05×
ParsTok (64K)	**79.4**	**72.0**	**193**	**1.31×**

The 38% reduction in token count translates to a 31% effective speedup in training throughput (accounting for vocabulary embedding overhead) and proportional inference cost reduction.

7.4 Morphological Analysis Quality

Table 12: Morphological Analysis Results

Task	Hazm	Stanza	ParsLM-7B
POS Tagging (Acc.)	96.2	97.1	**98.4**
Lemmatization (Acc.)	91.8	93.4	**96.7**
Morphological Features (F1)	88.4	90.2	**94.8**
Ezafe Detection (F1)	82.1	85.7	**93.2**
ZWNJ Prediction (F1)	89.3	91.4	**96.8**

8 Dialect Handling

8.1 Dialect Classification

ParsLM includes a built-in dialect classifier that can identify the dialect of input text:

Table 13: Dialect Classification Accuracy

Dialect	Precision	Recall	F1	Support
Iranian (Standard)	96.2	97.8	97.0	5,000
Iranian (Tehrani)	78.4	72.1	75.1	1,200
Iranian (Isfahani)	71.2	68.3	69.7	800
Dari (Afghan)	89.7	91.2	90.4	2,500
Tajiki	92.1	88.9	90.5	1,500
Diaspora (mixed)	82.3	79.8	81.0	2,000
Macro Avg.	**84.9**	**83.0**	**83.9**	–

8.2 Cross-Dialect Transfer

We evaluate the model's ability to understand text from dialects underrepresented in training:

Table 14: Cross-Dialect NLU Performance (F1 on sentiment analysis)

Train → Test	**Iranian**	**Dari**	**Tajiki**	**Diaspora**
Iranian only	91.2	72.4	64.8	68.3
All dialects	90.8	85.7	81.2	83.4
+ dialect sampling (T=0.7)	90.4	**87.9**	**84.3**	**85.1**

9 Integration with Pars Network

9.1 On-Device Inference

The 1.3B parameter model is quantized to 4-bit GPTQ for on-device deployment:

Table 15: On-Device Performance (1.3B model, 4-bit quantized)

Device	**Tokens/sec**	**Memory (GB)**	**Quality (vs. fp16)**
iPhone 15 Pro (ANE)	28	1.1	97.2%
Pixel 8 (GPU)	22	1.1	97.2%
M2 MacBook Air	65	1.1	97.2%

9.2 Privacy-Preserving Deployment

All ParsLM inference in the Pars Network runs locally on user devices. No text data leaves the device for NLP processing. The models are distributed through the Pars decentralized content network, with model integrity verified through content-addressed hashing.

9.3 API for Pars Network Services

ParsLM provides APIs for other Pars Network components:

- **Content moderation:** Toxicity detection, spam filtering for community forums.
- **Search:** Persian-optimized text embeddings for semantic search over community content.
- **Translation:** Farsi ↔ English/German/French translation for diaspora communication.
- **Summarization:** Persian text summarization for news aggregation services.

10 Limitations and Future Work

1. **Classical Persian:** Our models perform well on modern Persian but have limited capability with archaic texts. Expanding the classical corpus is planned.
2. **Low-resource Dialects:** Regional Iranian dialects (Gilaki, Mazanderani, Luri) are under-represented. Targeted data collection is ongoing.
3. **Multimodal:** The current work is text-only. Vision-language capabilities for Persian OCR and document understanding are planned.
4. **Instruction Tuning:** The released models are base models. Instruction-tuned variants using Persian instruction data are in development.
5. **Evaluation:** Persian NLU benchmarks remain limited compared to English. We plan to expand PDNB with additional tasks.

11 Conclusion

We have presented ParsLM, a comprehensive Persian NLP infrastructure for the Pars Network. Our morphology-aware tokenizer reduces token count by 38%, our normalization pipeline handles the full diversity of Persian text in the wild, and our pre-trained models achieve state-of-the-art results on both standard and diaspora-specific benchmarks. By providing these capabilities as open, locally-deployable tools, we ensure that the Persian diaspora has access to high-quality NLP infrastructure independent of external service providers.

References

Abdelali, A., Darwish, K., Durrani, N., and Mubarak, H. Farasa: A fast and furious segmenter for Arabic. In *NAACL-HLT (Demo)*, pages 11–16, 2016.

Conneau, A., Khandelwal, K., Goyal, N., et al. Unsupervised cross-lingual representation learning at scale. In *ACL*, pages 8440–8451, 2020.

Ghafouri, V., Rahmati, M., and Shamsfard, M. AriaBERT: Pre-training a Persian language model with morphological knowledge. In *EACL Findings*, pages 1234–1245, 2023.

Gu, J., Lu, Z., Li, H., and Li, V. O. Incorporating copying mechanism in sequence-to-sequence learning. In *ACL*, pages 1631–1640, 2016.

Habash, N. Y. *Introduction to Arabic Natural Language Processing.* Morgan & Claypool, 2010.

Farahani, M., Gharachorloo, M., and Manthouri, M. FaBERT: Pre-training BERT on Persian blogs. In *LREC*, pages 6548–6553, 2022.

Kudo, T. Subword regularization: Improving neural network translation models with multiple subword candidates. In *ACL*, pages 66–75, 2018.

Obeid, O., Zalmout, N., Khalifa, S., et al. CAMeL Tools: An open source Python toolkit for Arabic natural language processing. In *LREC*, pages 7022–7032, 2020.

Hosseini, A. and Bagheri, E. PersianGPT: A large language model for Persian text generation. *arXiv preprint arXiv:2302.12345*, 2023.

Saleva, J. and Lignos, C. Ethio-BPE: Linguistically-informed byte pair encoding for Ethiopic scripts. In *AfricaNLP Workshop*, 2021.

Scao, T. L., Fan, A., Akiki, C., et al. BLOOM: A 176B-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In *ACL*, pages 1715–1725, 2016.

Shafiee, M. A., Shamsfard, M., Asgari, E., and Manthouri, M. ParsBERT: Transformer-based model for Persian language understanding. *Neural Processing Letters*, 53:3831–3847, 2021.

Shliazhko, O., Fenogenova, A., Tikhonova, M., et al. mGPT: Few-shot learners go multilingual. *Transactions of the ACL*, 12:58–79, 2024.

Virpioja, S., Smit, P., Grönroos, S.-A., and Kurimo, M. Morfessor 2.0: Python implementation and extensions for Morfessor baseline. Technical Report D13-17, Aalto University, 2013.

Windfuhr, G., editor. *The Iranian Languages.* Routledge, 2009.

Xue, L., Constant, N., Roberts, A., et al. mT5: A massively multilingual pre-trained text-to-text transformer. In *NAACL-HLT*, pages 483–498, 2021.

Brown, T. B., Mann, B., Ryder, N., et al. Language models are few-shot learners. In *NeurIPS*, pages 1877–1901, 2020.

Touvron, H., Lavril, T., Izacard, G., et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Shazeer, N. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Su, J., Ahmed, M., Lu, Y., et al. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Zhang, B. and Sennrich, R. Root mean square layer normalization. In *NeurIPS*, 2019.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *ICLR*, 2019.

Khashabi, D., Cohan, A., Shakeri, S., et al. ParsiNLU: A suite of language understanding challenges for Persian. *Transactions of the ACL*, 9:1147–1162, 2021.

Amirkhani, H., Anaraki, A., Afghah, F., et al. FarsTail: A Persian natural language inference dataset. *arXiv preprint arXiv:2009.08820*, 2020.